

GROUP 15 MEMBERS:

ALI ABDULLAH

ASRA ANEES

FAIZAN SHABBIR

Laboratory 2: Monitoring VM Performance Counters

Task 1: Setup and run an amazon EC2 instance

1. Initializing a security group:

Begin by selecting the “Security Group” option located in the left-hand panel of the AWS console under services to create a new security group. Once there, specify a name and description, and set up the group’s inbound and outbound rules.

- **Security Group Details:**

- o Name: [MySecGroup15](#)
- o ID: [sg-087c73b057bbad445](#) [vpc-05c50d97582678a66](#)
- o Virtual private cloud ID: [vpc-05c50d97582678a66](#)
- o Description: [secgroup](#)
- o Inbound Rules: [All \(0.0.0.0/0\)](#), [SSH \(0.0.0.0/0\)](#), [HTTPS \(0.0.0.0/0\)](#) [IPV4](#)
- o Outbound Rules: [All \(0.0.0.0/0\)](#)

sg-087c73b057bbad445 - MySecGroup15 Actions ▾

Details

Security group name  MySecGroup15	Security group ID  sg-087c73b057bbad445	Description  secgroup	VPC ID  vpc-05c50d97582678a66 
Owner  645236540399	Inbound rules count 3 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (3) Refresh Manage tags Edit inbound rules

< 1 > ⊗

☐	Name ▾	Security group rule... ▾	IP version ▾	Type ▾	Protocol ▾	Port range ▾	Source ▾	Des
☐	-	sgr-02e6ff05b56a40a40	IPv4	All ICMP - IPv4	ICMP	All	0.0.0.0/0	-
☐	-	sgr-0be7493a606461...	IPv4	SSH	TCP	22	0.0.0.0/0	-
☐	-	sgr-0aa59f41670f4a0d4	IPv4	HTTPS	TCP	443	0.0.0.0/0	-

Outbound rules [Info](#)

Security group rule ID	Type Info	Protocol Info	Port range Info	Destination Info	Description - optional Info	
sg-027d794ff5e3248c1	All traffic ▾	All	All	Custom ▾ 0.0.0.0/0 		Delete

Add rule

Motivation: The motivation for using HTTPS, SSH, and ALL inbound rules is to ensure secure and reliable web services. These rules allow incoming traffic on the specified ports, which are essential for secure communication and remote access. The ALL outbound rule is used to allow all types of web services to be used at any time. This allows for flexibility and adaptability in the use of web services.

2. Establishing a Virtual Private Cloud (VPC):

Establish a Virtual Private Cloud (VPC) by navigating to the VPC service in the AWS console and following these steps:

Step1: Setting up Virtual private Cloud:

- ID: [vpc-0e26d591bcf1cfdcc](#)
- Main Route table: [rtb-0f7fc6ad26ae6419d](#)
- Main Network ACL: [acl-096aad918a625c034](#)

VPC > Your VPCs > vpc-0e26d591bcf1cfdcc			
vpc-0e26d591bcf1cfdcc / myvpcgroup15			
Actions ▼			
Details Info			
VPC ID vpc-0e26d591bcf1cfdcc	State Available	DNS hostnames Disabled	DNS resolution Enabled
Tenancy Default	DHCP option set dopt-0d70590e26345c3fb	Main route table rtb-0f7fc6ad26ae6419d	Main network ACL acl-096aad918a625c034
Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -	IPv6 CIDR (Network border group) -

Step2: Generating Subnet.

- ID: [subnet-0fb582c75e3884d82](#)
- Virtual private cloud: [vpc-0e26d591bcf1cfdcc](#) | [myvpcgroup15](#)
- Main Route table: [rtb-0f7fc6ad26ae6419d](#)
- Main Network ACL: [acl-096aad918a625c034](#)

Details			
Subnet ID subnet-0fb582c75e3884d82	Subnet ARN arn:aws:ec2:us-east-1:645236540399:subnet/subnet-0fb582c75e3884d82	State Available	IPv4 CIDR 10.0.0.0/16
Available IPv4 addresses 65531	IPv6 CIDR -	Availability Zone us-east-1a	Availability Zone ID use1-az2
Network border group us-east-1	VPC vpc-0e26d591bcf1cfdcc myvpcgroup15	Route table rtb-0f7fc6ad26ae6419d	Network ACL acl-096aad918a625c034
Default subnet No	Auto-assign public IPv4 address No	Auto-assign IPv6 address No	Auto-assign customer-owned IPv4 address No
Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -	IPv6 CIDR reservations -

Step3: Generating Route table.

- Main Route ID: [rtb-068651148c7605049](#)
- Virtual private cloud: [vpc-0e26d591bcf1cfdcc](#) | [myvpcgroup15](#)
- Route Table Name: [myroutetablegroup15](#)

rtb-068651148c7605049 / myroutetablegroup15 Actions ▾

🔍 You can now check network connectivity with Reachability Analyzer Run Reachability Analyzer ✕

Details [Info](#)

Route table ID rtb-068651148c7605049	Main No	Explicit subnet associations -	Edge associations -
VPC vpc-0e26d591bcf1cfdcc myvpcgroup15	Owner ID 645236540399		

Routes | Subnet associations | Edge associations | Route propagation | Tags

Routes (1) Edit routes

Both ▾ < 1 > ⌂

Destination ▾	Target ▾	Status ▾	Propagated ▾
10.0.0.0/16	local	Active	No

Step4: Generating Internet Gateway.

- ID: [igw-07306abc7c519f23f](#)
- Internet Gateway Name: [myinternetgatewaygroup15](#)

igw-07306abc7c519f23f / myinternetgatewaygroup15 Actions ▾

Details [Info](#)

Internet gateway ID igw-07306abc7c519f23f	State Detached	VPC ID -	Owner 645236540399
--	-------------------	-------------	-----------------------

Tags Manage tags

< 1 > ⌂

Key	Value
Name	myinternetgatewaygroup15

Setting up an EC2 Instance:

To set up an EC2 instance, we followed these steps:

1. Go to the EC2 service in the compute section of the AWS console.
2. Click on "[LaunchEC2](#)".
3. Give your instance a name, such as "[MyWebServiceGroup15](#)".
4. Select an image to use for the instance, such as "[Ubuntu](#)".
5. Follow the remaining prompts and configure settings as desired to launch your EC2 instance.

Name

MyWebServiceGroup15

Add additional tags

▼ Application and OS Images (Amazon Machine Image) Info

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Q Search our full catalog including 1000s of application and OS images

AMI from catalog

Quick Start

Amazon Machine Image (AMI)

ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20221201-ami-0574da719dca65348

Free tier eligible

Verified provider

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Catalog	Published	Architecture	Virtualization	Root device type	ENA Enabled
Quickstart AMIs	2022-12-01T15:56:50.00Z	x86_64	hvm	ebs	Yes

▼ Summary

Number of instances Info

1

Software Image (AMI)

Ubuntu Server 22.04 LTS (HVM),...read more
ami-0574da719dca65348

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet

Cancel

Launch instance

Select security group, network, and storage capacity and then click launch instance.

▼ Network settings Info

Edit

Network Info

vpc-023ff36eecd5ded97e

Subnet Info

No preference (Default subnet in any availability zone)

Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

We'll create a new security group called 'launch-wizard-4' with the following rules:

Allow SSH traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

×

▼ Summary

Number of instances Info

1

Ubuntu Server 22.04 LTS (HVM),...read more
ami-0574da719dca65348

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet

Cancel

Launch instance

Then wait for the next window to open and click connect.

Connect to instance Info

Connect to your instance i-0b3c6c83ff24badf3 (MyWebServiceGroup15) using any of these options

EC2 Instance Connect

Session Manager

SSH client

EC2 serial console

Instance ID

 i-0b3c6c83ff24badf3 (MyWebServiceGroup15)

Public IP address

 3.84.39.211

User name

Connect using a custom user name, or use the default user name ubuntu for the AMI used to launch the instance.

 **Note:** In most cases, the guessed user name is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI user name.

Cancel

Connect

Finally, the EC2 Image is launched and now we can push commands in this image.

```
aws Services Search [Alt+S]
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-1026-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

System information as of Fri Dec  9 18:11:42 UTC 2022

System load:  0.9384765625   Processes:           104
Usage of /:   19.8% of 7.57GB Users logged in:          0
Memory usage: 22%           IPv4 address for eth0: 172.31.94.201
Swap usage:   0%

0 updates can be applied immediately.

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

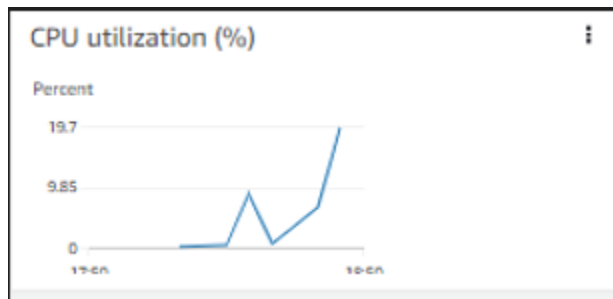
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-94-201:~$
```

Task 2: Monitoring VMs

CPU Utilization:



CPU utilization, Disk Read-Write operation, and Bytes in both EC2(Elastic Compute Cloud) and ESB (Elastic Block Store).

Load testing with “sysbench”:

Step1: Running: (“sudo sysbench --test=cpu --cpu-max-prime=200000 run”)

```
Initializing random number generator from current time

Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
  events per second:      13.96

General statistics:
  total time:              10.0242s
  total number of events:  140

Latency (ms):
  min:                     71.03
  avg:                     71.59
  max:                     76.47
  95th percentile:        73.13
  sum:                     10022.78

Threads fairness:
  events (avg/stddev):      140.0000/0.00
  execution time (avg/stddev): 10.0228/0.00

ubuntu@ip-172-31-80-22:~$
```

Step2: Running (“for each in 1 2 4 8 16 32 64; do sysbench --test=cpu --cpu-max-prime=200000 --num-threads=\$each run; done”)

Notice that “threads are increased from 1 to 64. The “sysbench” runs total of 7 times”.

- Running “sysbench” 7 times.
- Total time is taken for execution: 13.9136sec
- Per event speed of CPU: 13.80sec

```
aws Services Search
Initializing random number generator from current time

Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
  events per second:    13.80

General statistics:
  total time:           13.9136s
  total number of events: 192

Latency (ms):
  min:                  4363.22
  avg:                  4583.64
  max:                  4797.30
  95th percentile:     4683.57
  sum:                  880059.04

Threads fairness:
  events (avg/stddev):  3.0000/0.00
  execution time (avg/stddev): 13.7509/0.08
ubuntu@ip-172-31-80-22:~$
```

Step3: Running (“sysbench --test=fileio --file-total-size=8G --file-num=3 prepare”)

```
ubuntu@ip-172-31-80-22:~$ sysbench --test=fileio --file-total-size=8G --file-num=3 prepare
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

3 files, 2796202Kb each, 8191Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
Creating file test_file.1
Creating file test_file.2
8589950976 bytes written in 133.30 seconds (61.46 MiB/sec).
ubuntu@ip-172-31-80-22:~$
```

Step4: (“for each in 1 4 8 16 32; do sysbench --test=fileio --file-total-size=8G --file-test-mode=rndwr --max-time=240 --max-requests=0 --file-block-size=4K --file-num=3 --num-threads=\$each run; done;”)

```

ubuntu@ip-172-31-80-22:~$ for each in 1 4 8 16 32; do sysbench --test=fileio --file-total-size=8G --file-test-mode=rndwr --max-time=240 --max-requests=0 --file-block-size=4K --file-num=3 --num-threads=$each run; done;
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
WARNING: --max-time is deprecated, use --time instead
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time

Extra file open flags: (none)
3 files, 2.6667GiB each
8GiB total file size
Block size 4KiB
Number of IO requests: 0
Read/Write ratio for combined random IO test: 1.50
Periodic fsync enabled, calling fsync() each 100 requests.
Calling fsync() at the end of test, Enabled.
Using synchronous I/O mode
Doing random write test
Initializing worker threads...

Threads started!

```

Notice that the running command creates a single file and performs read & write operations 5 times for creating a load of CPU. The execution time taken by “sysbench” is 241.3788sec.

```

File operations:
  reads/s:           0.00
  writes/s:          44.30
  fsyncs/s:          1.52

Throughput:
  read, MiB/s:        0.00
  written, MiB/s:      0.17

General statistics:
  total time:          241.3788s
  total number of events: 11012

Latency (ms):
  min:                 0.00
  avg:                 349.14
  max:                 2740.20
  95th percentile:    1191.92
  sum:                 3844687.67

Threads fairness:
  events (avg/stddev): 688.2500/25.60
  execution time (avg/stddev): 240.2930/0.18

```

Motivation: The motivation for using a workload generator such as sysbench on AWS is to simulate a realistic load on a system in order to test its performance and scalability within the AWS environment. Sysbench is a multi-threaded benchmark tool that can be used to test various system parameters such as CPU, memory, disk, and network performance.

Task 3: Communication

Load testing with “apachebench”:

Virtual Machine 1:

- ab -n 5500 -c 250: <http://172.31.90.202/html>
- IP address: 172.31.90.202

- Request: 5500
- Client: 250
- Server Port: 80
- Transfer rate: 2119.20 [Kbytes/sec]
- Time per request: 52.418[ms]

```

0 updates can be applied immediately.

Last login: Mon Dec 12 01:36:01 2022 from 18.206.107.28
ubuntu@ip-172-31-81-116:~$ ab -n 5500 -c 250 http://172.31.90.202/html
This is ApacheBench, Version 2.3 <699-1310> 1879480 <S>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.31.90.202 (be patient)
Completed 550 requests
Completed 1100 requests
Completed 1650 requests
Completed 2200 requests
Completed 2750 requests
Completed 3300 requests
Completed 3850 requests
Completed 4400 requests
Completed 4950 requests
Completed 5500 requests
Finished 5500 requests


Server Software:      Apache/2.4.18
Server Hostname:      172.31.90.202
Server Port:          80

Document Path:        /html
Document Length:       275 bytes

Concurrency Level:     250
Time taken for tests:   1.133 seconds
Complete requests:     5500
Failed requests:        0
Non-2xx responses:      5500
Total transferred:      2502500 bytes
HTML transferred:       1512500 bytes
Requests per second:    4765.36 [#/sec] (mean)
Time per request:       52.418 [ms] (mean)
Time per request:       0.210 [ms] (mean, across all concurrent requests)
Transfer rate:          2119.20 [Kbytes/sec] received


Connection Times (ms)
  min      mean[+/-sd] median max
Connect:    0        6  4.1  7   26
Processing:  3      43  8.2  44  64
Waiting:    1      43  8.2  44  63
Total:      11      51  7.3  52  72


Percentage of the requests served within a certain time (ms)
 50%    32
 66%    34
 75%    36
 80%    36
 90%    36
 95%    39
 98%    44
 99%    46
100%    72 (longest request)
ubuntu@ip-172-31-81-116:~$

```

i-025e664e56e744151 (VM 1)

PublicIP: 44.202.0.251 PrivateIP: 172.31.81.116

Virtual Machine 2:

- ab -n 5500 -c 250: [http:// 172.31.81.116/html](http://172.31.81.116/html)
- IP address: 172.31.81.116
- Request: 5500
- Client: 250
- Server Port: 80
- Transfer rate: 1967.22[Kbytes/sec]
- Time per request: 56.468[ms]

```
0 updates can be applied immediately.

Last login: Mon Dec 12 01:38:34 2022 from 18.206.107.27
ubuntu@ip-172-31-50-202:~$ ab -n 5500 -c 250 http://172.31.81.116/html
This is ApacheBench, Version 2.3 <69913141> 1879480 <6>
Copyright 1996 Adex T.iao, Teas Technology Ltd, http://www.teas-tech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.31.81.116 (be patient)
Completed 550 requests
Completed 1100 requests
Completed 1650 requests
Completed 2200 requests
Completed 2750 requests
Completed 3300 requests
Completed 3850 requests
Completed 4400 requests
Completed 4950 requests
Completed 5500 requests
Finished 5500 requests

Server Software:      Apache/2.4.52
Server Hostname:      172.31.81.116
Server Port:          80

Document Path:        /html
Document Length:       275 bytes

Concurrency Level:     250
Time taken for tests:   1.242 seconds
Complete requests:     5500
Failed requests:        0
Non-2xx responses:     5500
Total transferred:     2502500 bytes
HTML transferred:     1512500 bytes
Requests per second:   4427.31 [#/sec] (mean)
Time per request:      56.468 [ms] (mean)
Time per request:      0.226 [ms] (mean, across all concurrent requests)
Transfer rate:         1967.22 [Kbytes/sec] received

Connection Times (ms)
  min      mean[+/-sd] median max
Connect:    0       7   4.5      7    21
Processing:  6      48   9.3     49   256
Waiting:    1      48   9.4     49   256
Total:      10     55   9.5     56   261

Percentage of the requests served within a certain time (ms)
 50%    56
 66%    58
 75%    60
 80%    61
 90%    64
 95%    66
 98%    68
 99%    70
100%   261 (longest request)
ubuntu@ip-172-31-50-202:~$
```

i-0f9414bd618f3a67a (VM 2)

PublicIP: 3.86.104.231 PrivateIP: 172.31.50.202

For connection:

- Installed in Virtual Machine 1: [sudo apt-get install apache](#)
- Installed in Virtual Machine 2: [sudo apt-get install -y apache2-utils](#)

Used ping and wget for communication between these Instances i.e Virtual Machine 1 and Virtual Machine 2.

Ping from Virtual Machine 1 to Virtual Machine 2:

```
ubuntu@ip-172-31-81-116:~$ ping 172.31.90.202
PING 172.31.90.202 (172.31.90.202) 56(84) bytes of data.
64 bytes from 172.31.90.202: icmp_seq=1 ttl=64 time=0.614 ms
64 bytes from 172.31.90.202: icmp_seq=2 ttl=64 time=0.578 ms
64 bytes from 172.31.90.202: icmp_seq=3 ttl=64 time=0.999 ms
64 bytes from 172.31.90.202: icmp_seq=4 ttl=64 time=0.631 ms
64 bytes from 172.31.90.202: icmp_seq=5 ttl=64 time=0.627 ms
64 bytes from 172.31.90.202: icmp_seq=6 ttl=64 time=3.54 ms
64 bytes from 172.31.90.202: icmp_seq=7 ttl=64 time=0.496 ms
64 bytes from 172.31.90.202: icmp_seq=8 ttl=64 time=0.613 ms
64 bytes from 172.31.90.202: icmp_seq=9 ttl=64 time=0.528 ms
64 bytes from 172.31.90.202: icmp_seq=10 ttl=64 time=0.595 ms
64 bytes from 172.31.90.202: icmp_seq=11 ttl=64 time=0.583 ms
64 bytes from 172.31.90.202: icmp_seq=12 ttl=64 time=0.637 ms
64 bytes from 172.31.90.202: icmp_seq=13 ttl=64 time=0.574 ms
64 bytes from 172.31.90.202: icmp_seq=14 ttl=64 time=0.584 ms
64 bytes from 172.31.90.202: icmp_seq=15 ttl=64 time=0.539 ms
64 bytes from 172.31.90.202: icmp_seq=16 ttl=64 time=0.831 ms
64 bytes from 172.31.90.202: icmp_seq=17 ttl=64 time=0.573 ms
64 bytes from 172.31.90.202: icmp_seq=18 ttl=64 time=0.579 ms
64 bytes from 172.31.90.202: icmp_seq=19 ttl=64 time=0.576 ms
64 bytes from 172.31.90.202: icmp_seq=20 ttl=64 time=0.570 ms
^C
--- 172.31.90.202 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19367ms
rtt min/avg/max/mdev = 0.496/0.763/3.536/0.645 ms
ubuntu@ip-172-31-81-116:~$
```

i-025e664e56e744151 (VM 1)

PublicIPs: 44.202.0.251 PrivateIPs: 172.31.81.116

From Virtual Machine 1 (wget):

```
ubuntu@ip-172-31-81-116:~$ wget 172.31.90.202
--2022-12-12 02:09:48-- http://172.31.90.202/
Connecting to 172.31.90.202:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10671 (10K) [text/html]
Saving to: 'index.html'

index.html
index.html
=====>] 10.42K --.-KB/s in 0s

2022-12-12 02:09:48 (192 MB/s) - 'index.html' saved [10671/10671]
ubuntu@ip-172-31-81-116:~$
```

i-025e664e56e744151 (VM 1)

PublicIPs: 44.202.0.251 PrivateIPs: 172.31.81.116

Ping from Virtual Machine 2 to Virtual Machine 1:

```
100% 201 (longest request)
ubuntu@ip-172-31-90-202:~$ ping 172.31.81.116
PING 172.31.81.116 (172.31.81.116) 56(84) bytes of data:
64 bytes from 172.31.81.116: icmp_seq=1 ttl=64 time=0.525 ms
64 bytes from 172.31.81.116: icmp_seq=2 ttl=64 time=0.584 ms
64 bytes from 172.31.81.116: icmp_seq=3 ttl=64 time=0.540 ms
64 bytes from 172.31.81.116: icmp_seq=4 ttl=64 time=0.509 ms
64 bytes from 172.31.81.116: icmp_seq=5 ttl=64 time=0.572 ms
64 bytes from 172.31.81.116: icmp_seq=6 ttl=64 time=0.550 ms
64 bytes from 172.31.81.116: icmp_seq=7 ttl=64 time=0.556 ms
64 bytes from 172.31.81.116: icmp_seq=8 ttl=64 time=0.532 ms
64 bytes from 172.31.81.116: icmp_seq=9 ttl=64 time=0.607 ms
64 bytes from 172.31.81.116: icmp_seq=10 ttl=64 time=0.679 ms
64 bytes from 172.31.81.116: icmp_seq=11 ttl=64 time=0.568 ms
64 bytes from 172.31.81.116: icmp_seq=12 ttl=64 time=0.677 ms
64 bytes from 172.31.81.116: icmp_seq=13 ttl=64 time=0.570 ms
64 bytes from 172.31.81.116: icmp_seq=14 ttl=64 time=0.630 ms
64 bytes from 172.31.81.116: icmp_seq=15 ttl=64 time=0.610 ms
64 bytes from 172.31.81.116: icmp_seq=16 ttl=64 time=0.603 ms
64 bytes from 172.31.81.116: icmp_seq=17 ttl=64 time=0.600 ms
64 bytes from 172.31.81.116: icmp_seq=18 ttl=64 time=0.647 ms
64 bytes from 172.31.81.116: icmp_seq=19 ttl=64 time=0.759 ms
64 bytes from 172.31.81.116: icmp_seq=20 ttl=64 time=0.686 ms
64 bytes from 172.31.81.116: icmp_seq=21 ttl=64 time=0.575 ms
64 bytes from 172.31.81.116: icmp_seq=22 ttl=64 time=0.624 ms
64 bytes from 172.31.81.116: icmp_seq=23 ttl=64 time=0.778 ms
64 bytes from 172.31.81.116: icmp_seq=24 ttl=64 time=0.561 ms
64 bytes from 172.31.81.116: icmp_seq=25 ttl=64 time=0.622 ms
^C
--- 172.31.81.116 ping statistics ---
25 packets transmitted, 25 received, 0% packet loss, time 24563ms
rtt min/avg/max/mdev = 0.509/0.606/0.778/0.066 ms
ubuntu@ip-172-31-90-202:~$
```

i-0f9414bd618f3a67a (VM 2)

PublicIPs: 3.86.104.231 PrivateIPs: 172.31.90.202

From Virtual Machine 2 (wget):

```
rtt min/avg/max/mdev = 0.509/0.606/0.778/0.066 ms
ubuntu@ip-172-31-90-202:~$ wget 172.31.81.116
--2022-12-12 02:12:51-- http://172.31.81.116/
Connecting to 172.31.81.116:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10671 (10K) [text/html]
Saving to: 'index.html'

index.html
index.html
=====>] 10.42K --.-KB/s in 0s

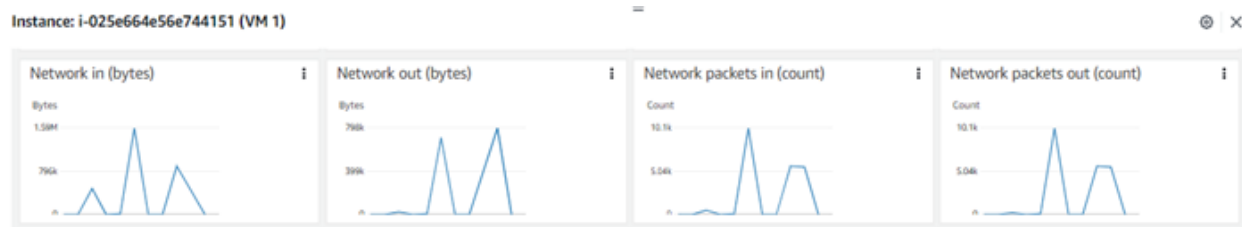
2022-12-12 02:12:51 (210 MB/s) - 'index.html' saved [10671/10671]
ubuntu@ip-172-31-90-202:~$
```

i-0f9414bd618f3a67a (VM 2)

PublicIPs: 3.86.104.231 PrivateIPs: 172.31.90.202

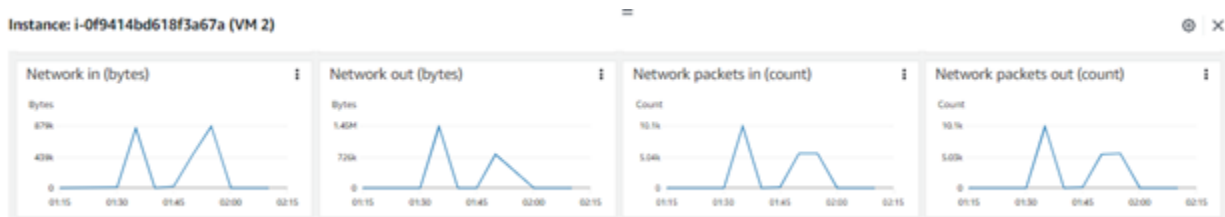
In Virtual Machine 1 graph:

- Network-in: 1.59m
- Network-out: 798k
- Network packets-in: 10.1k
- Network packets-out: 10.1k



In Virtual Machine 2 graph:

- Network-in: 879k
- Network-out: 1.45m
- Network packets-in: 10.1k
- Network packets-out: 10.1k



Motivation: We used to connect two virtual machines using ApacheBench (ab) - 5500 requests, 250 clients, and server port 80 - were chosen to simulate a realistic load on the web server and evaluate its performance and scalability under high load conditions. By generating 5500 requests and using 250 clients to connect to the server, the load on the web server is increased significantly, which is useful for identifying bottlenecks and performance issues. Port 80 is the default port for HTTP, making it a commonly used port for web servers, and therefore, an ideal choice for testing.